

Guillermo Román

guillermo.roman@upm.es

Lars-Åke Fredlund

lfredlund@fi.upm.es

Manuel Carro

mcarro@fi.upm.es

Julio García

juliomanuel.garcia@upm.es

Tonghong Li

tonghong@fi.upm.es

- Fechas de entrega y penalización:

Hasta el Martes 15 de octubre, 16:00 horas	0
Hasta el Miércoles 16 de octubre, 16:00 horas	20 %
Hasta el Jueves 7 17 octubre, 16:00 horas	40 %

Después la puntuación máxima será 0
- Se comprobará plagio y se actuará sobre los detectados.
- Usad las horas de tutoría para preguntar sobre programación – son oportunidades excelentes para aprender.

Entrega

- Todos los ejercicios de laboratorio se deben entregar a través de
`http://deliverit.fi.upm.es`
- El/los fichero(s) que hay que subir es/son `OrderedIterator.java`.
- La clase debe estar en el paquete `aed.individual3`.
- La documentación de la API de `aedlib.jar` está disponible en
`http://costa.ls.fi.upm.es/teaching/aed/docs/aedlib/`

Configuración previa

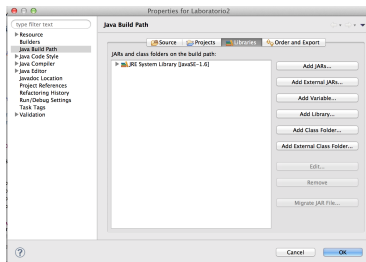
- Arrancad Eclipse
- Si trabajáis en portátil, podéis utilizar cualquier versión reciente de Eclipse. Es suficiente con que instaléis la *Eclipse IDE for Java Developers*.
- Cambiad a “Java Perspective”.
- Debéis tener instalado al menos Java JDK 8.
- Cread un proyecto Java llamado aed:
 - ▶ Seleccionad separación de directorios de fuentes y binarios.
 - ▶ **No debéis elegir la opción de crear el fichero** `module-info.java`
- Cread un *package* `aed.individual3` en el proyecto aed, dentro de `src`
- Moodle → AED → Practicas → Individual 4 → Individual4.zip; descomprimidlo
- Contenido de Individual4.zip:
 - ▶ `OrderedIterator.java`, `TesterInd4.java`

Configuración previa

- Importad al paquete `aed.individual3` los fuentes que habéis descargado (`OrderedIterator.java`, `TesterInd4.java`)
- Añadid al proyecto `aed` la librería `aedlib.jar` que tenéis en Moodle (en Laboratorios).

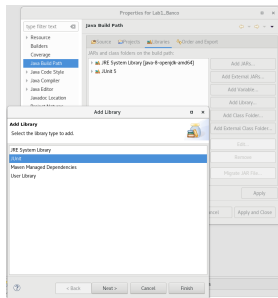
Para ello:

- Project → Properties → Java Build Path. Se abrirá una ventana como la de la izquierda
- Usad la opción “Add External JARs...”
- Si vuestra instalación distingue `ModulePath` y `ClassPath`, instalad en `ClassPath`



Configuración previa

- Añadid al proyecto aed la librería JUnit 5



Para ello:

- Project → Properties → Java Build Path. Se abrirá una ventana como la de la izquierda;
 - Usad la opción “Add Library...” → Seleccionad “JUnit” → Seleccionad “JUnit 5”
 - Si vuestra instalacion distingue ModulePath y ClassPath, instalad en ClassPath
-
- En la clase TesterInd4 tenéis las pruebas, para ejecutarlas, abrid el fichero TesterInd4, pulsando el botón derecho sobre el editor, seleccionar “Run as...” → “JUnit Test”
 - NOTA: Si al ejecutar, no aparece la vista “JUnit”, podéis incluirla en “Window” → “Show View” → “Java” → “JUnit”

Documentación de la librería aedlib.jar

- La documentación de la API de aedlib.jar está disponible en <http://costa.ls.fi.upm.es/teaching/aed/docs/aedlib/>
- También se puede añadir la documentación de la librería a Eclipse (*no es obligatorio*):
 - ▶ En el “Package Explorer”: “Referenced Libraries” → aedlib.jar y elige la opción “Properties”. Se abre una ventana donde se puede elegir “Javadoc Location” y ahí se pone como “javadoc location path:”

<http://costa.ls.fi.upm.es/teaching/aed/docs/aedlib/>
y presionar el botón “Apply and Close”

Tarea: Implementar un Iterador

- Termina la implementación de la clase `OrderedIterator` dentro el fichero `OrderedIterator.java`.
- Como la clase implementa la interfaz `Iterator<Integer>` debeis implementar obligatoriamente los métodos de un iterador: `hasNext`, `next`. No hace falta implementar `remove`.
- El constructor de dicha clase recibe un `PositionList<Integer> list` como argumento; los métodos del iterador debe devolver solamente los elementos dentro `list` que forman una secuencia *ordenada*.
- Concretamente, el primer entero dentro `list` siempre sera el primer elemento devuelto por `next()`. El segundo elemento devuelto por `next()` sera el siguiente entero dentro `list` que es *mayor o igual que el primer entero*, etc.

Ejemplos

list	elementos devueltos por el iterador

[]	
[1]	1
[1,1]	1,1
[2,1]	2
[1,2,3]	1,2,3
[5,1,2,3,4,6,7,2]	5,6,7
[-1,0,-2]	-1,0

- **Se debe implementar un iterador eficiente que no crea otras estructuras de datos.**
- Es permitido, y necesario, de añadir nuevos atributos dentro la clase `OrderedIterator`.
- Notad que `list` no contiene elementos `null`.

Notas Generales

- El proyecto debe compilar sin errores y debe cumplirse la especificación de los métodos a completar
- Debe pasar todos los test `TesterInd4` correctamente sin mensajes de error
- **Nota:** una ejecución sin mensajes de error y que pase todas las pruebas **no** significa que la implementación sea correcta (es decir, que funcione bien para cada posible entrada)
- Todos los ejercicios se corrigen manualmente antes de dar la nota final